

Title Here

XXXX

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Computing methodologies** → *Artificial intelligence*; • **Software and its engineering** → **Software libraries and repositories**.

Additional Key Words and Phrases: malicious package detection, open source packages, behavior sequence modeling, large language models

ACM Reference Format:

. 2018. Title Here. 1, 1 (February 2018), 14 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

XXXX

2 THREAT MODEL

XXXX

3 PRELIMINARY AND MOTIVATION

XXXX

4 METHODOLOGY

XXXX

5 EVALUATION

We first present the evaluation setup, and then report the evaluation results of research questions.

5.1 Evaluation Setup

Research Questions. We design our evaluation to answer the following four research questions.

- **RQ1 Effectiveness Evaluation:** How is the effectiveness of CEREPRO, compared with the state-of-the-art malicious package detection approaches?
- **RQ2 Efficiency Evaluation:** How is the performance overhead of CEREPRO?
- **RQ3 Dataset Scale Evaluation:** How does the scale of the dataset affect the effectiveness of CEREPRO?
- **RQ4 Ablation Study:** How behavior sequence contributes to the effectiveness of CEREPRO?
- **RQ5 Usefulness Evaluation:** How useful is CEREPRO in real-world detection on PyPI and NPM?

Author's address:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Table 1. Statistics of the Dataset

Package Registry	Malicious			Benign		
	# Packages	Aver. Files #	Aver. KLOC #	# Packages	Aver. Files #	Aver. KLOC #
PyPI	887	5.6	0.987	2,398	75.3	19.492
NPM	1,788	4.4	1.642	4,993	214.3	9.256
Mixed	2,675	4.8	1.425	7,391	53.4	12.577

Dataset Collection. We constructed the dataset as follows. For malicious packages, we collected malicious samples from three sources. We collected 438 malicious PyPI packages and 1,788 malicious NPM packages from Backstabber’s Knife Collection [37]. We added 88 malicious PyPI packages from MALOSS’s dataset [6]. We also sampled 361 packages out of a total of 5,874 packages from pypi_malregistry [54], with a confidence level of 95% and a margin of error of 5%. For benign packages, we collected 2,398 benign PyPI packages from Vu et al.’s dataset [50]. Following the procedure in prior work [35, 50], we selected the 5,000 most depended upon packages in the NPM registry as the dataset of benign NPM packages. However, we successfully downloaded 4,993 of them. In total, we curated a dataset of PyPI and NPM packages with 2,314 malicious packages and 7,391 benign packages, as listed in Table 1 where *Mixed* denotes the summation across PyPI and NPM. We also provide the average file count and average thousand lines of code (KLOC) for each package.

RQ Setup. For **RQ1**, we aim to compare the effectiveness of CEREbro with state-of-the-art approaches. Specifically, we selected AMALFI [42], SAP [26] and MPHUNTER [55] as the state-of-the-art learning-based approaches, and OSS Detect Backdoor [33] and Bandit4Mal [48] as the state-of-the-art rule-based approaches. We adopted the same configuration used in the original paper [26, 42, 55]. For OSS Detect Backdoor and Bandit4Mal, we set the threshold of 3 alerts to distinguish malicious and benign packages, which was observed as the optimal threshold by Vu et al. [50]. We did not compare with MALOSS [6] as it used dynamic features and thus failed to run on many packages. We also did not compare with Malware Checks [53] as the PyPI team removed it recently. We split the dataset into training and testing dataset by 9:1. We measured the precision and recall of all the approaches with 10-fold cross-validation on the testing datasets. We compared these approaches in three detection scenarios, i.e., mono-lingual scenario (i.e., train and test on the same ecosystem), cross-lingual scenario (i.e., train on one ecosystem and test on the other), and bi-lingual scenario (i.e., train on two ecosystems and test on one of the two ecosystems). We trained CEREbro with BERT [21], RoBERTa [32] and T5 [40] on the PyPI, NPM and Mixed training datasets, denoted by CEREbro_{BERT}, CEREbro_{RoBERTa} and CEREbro_{T5}. We also trained AMALFI with decision tree (AMALFI_{DT}), naive bayes (AMALFI_{NB}) and SVM (AMALFI_{SVM}) on the same training datasets. We only evaluate SAP in both mono-lingual and bi-lingual scenarios, and MPHUNTER in the mono-lingual scenario for PyPI packages, as their do not support adaptation to other scenarios.

For **RQ2**, we measured the time overhead of CEREbro in offline training and online prediction. For **RQ3**, we evaluated the effectiveness of CEREbro across different dataset scales. We incrementally increased the proportion of our training dataset from 10% to 100% in 10% steps. We trained CEREbro and AMALFI_{DT} in the bi-lingual scenario using these varying dataset proportions. The trained models were then tested using the same testing dataset across all scales. The average F1-Score over 10-fold cross-validation was used to measure overall effectiveness.

For **RQ4**, we created three ablated versions of CEREbro and compared their effectiveness with the original version of CEREbro using the same dataset in **RQ1**. Specifically, to evaluate the impact of sequential behavior, we created “CEREbro w/o Seq” by removing behavior sequence generator and feeding textual descriptions directly into maliciousness classifier. To evaluate the impact of textual description transformation, we created “CEREbro w/o Text” by removing textual

Table 2. Evaluation Result in Mono-Lingual Scenario for Learning-based Approaches

Train	Test	CEREBRO <i>BERT</i>		CEREBRO <i>RoBERTa</i>		CEREBRO <i>T5</i>		AMALFI <i>DT</i>		AMALFI <i>NB</i>		AMALFI <i>SVM</i>		SAP		MPHUNTER	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
PyPI	PyPI	95.8%	89.4%	96.0%	91.7%	93.4%	70.9%	81.9%	83.9%	59.5%	7.5%	30.3%	30.9%	83.9%	61.5%	21.8%	85.4%
NPM	NPM	98.2%	91.8%	98.5%	92.9%	98.9%	91.0%	92.7%	86.0%	92.1%	77.1%	10.2%	26.7%	90.5%	35.4%	–	–

Table 3. Evaluation Result in Mono-Lingual Scenario for Rule-Based Approaches

Test	OSS Detect Backdoor		Bandit4Mal	
	Pre.	Rec.	Pre.	Rec.
PyPI	19.9%	60.4%	24.9%	74.4%
NPM	41.1%	85.1%	–	–

description transformation in maliciousness classifier. To evaluate the impact of pre-trained language models, we created “CEREBRO w/ DT” by removing behavior sequence generator and feeding features as a vector into the decision tree.

For **RQ5**, we ran CEREBRO using the BERT model against the newly published packages in PyPI and NPM for over nine months. This monitoring process started on March 03 2023 for PyPI and April 01 2023 for NPM, and ended at October 30 2023. CEREBRO analyzed 599,493 PyPI package versions and 324,145 NPM package versions. We manually confirmed the potentially malicious package versions flagged by CEREBRO. Specifically, two authors manually assessed the maliciousness of the packages in the dimension of static characteristics, dynamic behaviors and third-party detection tools. Two authors conducted a manual assessment to determine the maliciousness of the packages. This process covered comprehending the source code, inspecting the dynamic behaviors, and referencing detection results from third-party detection tools (e.g., virustotal [47]). The two authors are postgraduate and undergraduate students with security/malware background. The human inspection was conducted daily, costing one hour each day, over a period of ten months. The average number of reviewed packages daily was 30. All packages confirmed by two authors as malicious were reported to the official PyPI and NPM teams, all of which were accepted by the two registries. Further, by adding the confirmed malicious and benign packages into the original training datasets (i.e., incremental learning), we retrained CEREBRO using the BERT model to evaluate whether CEREBRO could learn from new data to improve its effectiveness. We also analyzed the malicious intentions and triggering scenarios of the new malicious packages.

$$\begin{aligned}
 Pre. &= \frac{|MP_{tool} \cap MP_{test}|}{|MP_{tool}|} \\
 Rec. &= \frac{|MP_{tool} \cap MP_{test}|}{|MP_{test}|} \\
 F1 - Score &= \frac{2 \times Pre. \times Rec.}{Pre. + Rec.}
 \end{aligned} \tag{1}$$

Evaluation Metric. The precision (Pre.), recall (Rec.) used in **RQ1**, **RQ4**, **RQ5** and F1-Score used in **RQ3** are defined in Equation 1. Specifically, MP_{tool} denotes the set of malicious packages predicted by tools (e.g., CEREBRO), and MP_{test} denotes the set of true malicious packages in the testing dataset. Precision is calculated by the proportion of true positives (i.e., $MP_{tool} \cap MP_{test}$) in the predicted malicious packages (i.e., MP_{tool}), and recall is calculated by the proportion of true positives (i.e., $MP_{tool} \cap MP_{test}$) in the ground truth (i.e., MP_{test}).

5.2 Effectiveness Evaluation (RQ1)

Mono-Lingual Scenario. The result of learning-based approaches in mono-lingual scenario is reported in Table 2 and the result of rule-based approaches is reported in Table 3. Overall, CEREBRO *RoBERTa* outperforms all the other approaches. In PyPI, CEREBRO *RoBERTa* achieves a precision of 96.0% and a recall of 91.7%. It achieves the best result, with slight advantages compared with CEREBRO *BERT* and CEREBRO *T5*. It outperforms the state-of-the-art SAP by 12.1%

in precision, and MPHUNTER by 6.3% in recall. In NPM, CEREbro *RoBERTa* achieves a precision of 98.5% and a recall of 92.9%. It outperforms CEREbro *T5* with a 1.9% higher recall and a 0.4% lower precision, and CEREbro *BERT* with a 0.3% higher precision and 1.1% higher recall. It outperforms the state-of-the-art AMALFI_{DT} by 5.8% in precision and 6.9% in recall. The two rule-based approaches have low precision and recall.

Cross-Lingual Scenario. The result of cross-lingual scenario is presented in Table 4. When the model is trained on NPM packages and tested on PyPI packages, CEREbro *RoBERTa* obtains a highest precision of 75.1%. However, CEREbro *T5* achieves a more balanced performance with a precision of 65.2% and a recall of 63.4%. Although AMALFI_{SVM} has better recall, CEREbro *T5* significantly surpasses it in precision, resulting in an overall F1-Score advantage of 13.3%. When the model is trained on PyPI packages and tested on NPM packages, CEREbro *RoBERTa* obtains a highest precision of 46.5%, and a recall of 92.7%. Although AMALFI_{SVM} has better recall, CEREbro *RoBERTa* significantly surpasses it in precision, resulting in a F1-Score advantage of 17.7%. This result potentially owes to our well-abstracted feature set. Notice that CEREbro in the cross-lingual scenario achieves a lower precision and recall than in the mono-lingual scenario, which indicates that some malicious behaviors might be not common, and thus bi-lingual knowledge fusing is necessary.

Bi-Lingual Scenario. The result of bi-lingual scenario is shown in Table 5. Overall, CEREbro *RoBERTa* outperforms all the other approaches in PyPI and NPM packages. For testing PyPI packages using the model trained on mixed packages, CEREbro *BERT* obtains a precision of 95.0% and a recall of 92.0%, and CEREbro *RoBERTa* obtains a precision of 96.1% and a recall of 90.9%. They achieve very similar results, with CEREbro *RoBERTa* outperforming CEREbro *BERT* by 1.1% in precision, while CEREbro *BERT* surpasses CEREbro *RoBERTa* by the same margin in recall. Comparing with the state-of-the-art, CEREbro *RoBERTa* outperform AMALFI_{DT} by 12.9% in precision and 9.8% in recall. For testing NPM packages using the model trained on mixed packages, CEREbro *RoBERTa* achieves a precision of 98.9% and a recall of 93.9%. It outperforms CEREbro *BERT* by 0.2% in precision and 0.9% in recall, and surpasses CEREbro *T5* by 1.9% in recall. Comparing with the state-of-the-art, CEREbro *RoBERTa* outperforms AMALFI_{DT} by 6.8% in precision 8.1% in recall. Although AMALFI_{SVM} outperforms CEREbro in recall by 0.7%, it suffers from a very low precision of 28.8%.

Comparing Bi-lingual with Mono-lingual. Comparing the effectiveness of CEREbro in bi-lingual and mono-lingual scenarios, we observe that CEREbro *BERT*, CEREbro *RoBERTa*, and CEREbro *T5* trained on mixed packages outperform their mono-lingual counterparts. For detecting malicious PyPI packages, the bi-lingual model of CEREbro achieves an average precision increase of 0.2% and an average recall increase of 0.2%. For NPM packages, the bi-lingual model achieves an average precision increase of 0.3% and an average recall increase of 1.1%. These results indicate that our bi-lingual knowledge fusion is useful.

When comparing the performance of CEREbro *BERT* and CEREbro *RoBERTa*, we observe that CEREbro *RoBERTa* generally outperforms CEREbro *BERT* in most scenarios. This advantage can be attributed to RoBERTa’s use of more advanced training strategies, including dynamic masking, larger batch sizes, and an increased number of training steps, which enable the model to learn more efficiently. However, in cross-lingual and bi-lingual tasks, CEREbro *BERT* achieves higher or comparable recall compared to CEREbro *RoBERTa*. Further analysis reveals that for malicious packages detected by CEREbro *BERT* but missed by CEREbro *RoBERTa*, the sequences tend to be longer. This suggests that CEREbro *RoBERTa* may be less effective at capturing long-range dependencies within longer sequences, which are essential for recognizing complex malicious patterns. A potential explanation for this discrepancy lies in the pre-training tasks of the two models: RoBERTa does not include the Next Sentence Prediction (NSP) task during its training, while BERT does. The inclusion of NSP in BERT’s training may enhance its ability to comprehend long-context relationships, making it better at detecting certain malicious packages that RoBERTa misses.

Table 4. Evaluation Result in Cross-Lingual Scenario

Train	Test	CEREBRO <i>BERT</i>		CEREBRO <i>RoBERTa</i>		CEREBRO <i>T5</i>		<i>AMALFiDT</i>		<i>AMALFiNB</i>		<i>AMALFiSVM</i>	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
NPM	PyPI	72.0%	49.1%	75.1%	44.1%	65.2%	63.4%	72.0%	18.0%	42.0%	31.5%	38.4%	76.0%
PyPI	NPM	41.9%	93.0%	46.5%	92.7%	37.5%	90.9%	14.4%	4.7%	48.8%	5.2%	28.8%	94.6%

Table 5. Evaluation Result in Bi-Lingual Scenario

Train	Test	CEREBRO <i>BERT</i>		CEREBRO <i>RoBERTa</i>		CEREBRO <i>T5</i>		<i>AMALFiDT</i>		<i>AMALFiNB</i>		<i>AMALFiSVM</i>		SAP	
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.
Mixed	PyPI	95.0%	92.0%	96.1%	90.9%	94.6%	69.6%	83.2%	81.1%	42.0%	35.6%	45.1%	37.2%	80.2%	60.2%
	NPM	98.7%	93.0%	98.9%	93.9%	98.9%	92.0%	92.1%	85.8%	93.9%	76.6%	28.8%	94.6%	90.4%	39.5%

Table 6. Efficiency Evaluation Results

Training Data	Training Time (h)	Average Prediction Time (s)			
		Feature Extractor	Behavior Sequence Generator	Maliciousness Classifier	Total
PyPI	14.1	3.737	7.807	0.008	11.552
NPM	25.2	2.412	7.611	0.007	10.030
Mixed	39.3	2.856	7.664	0.007	10.527

Summary. CEREBRO *RoBERTa* outperforms the state-of-the-art by averagely 10.0% in precision and 7.4% in recall in the mono-lingual scenario, and averagely 9.9% in precision and 8.9% in recall in the bi-lingual scenario. CEREBRO learns bi-lingual knowledge from two package registries, which contributes to an average increase of 0.3% in precision and 0.7% in recall.

5.3 Efficiency Evaluation (RQ2)

We measure the time overhead of CEREBRO for training and prediction, including all components described in Section 4. CEREBRO is trained on a machine with an Intel Xeon(R) Silver 4314 CPU at 2.40GHz, 128G of RAM and an NVIDIA GeForce RTX 3090. The same machine is employed for predictions. Table 6 presents the efficiency results. To train CEREBRO with packages from a single package registry, it takes around 14.1 hours for PyPI packages and 25.2 hours for NPM packages. To equip CEREBRO with bi-lingual knowledge using the mixed packages, the training time is proportional to the scale of the dataset, which amounts to 39.3 hours. Moreover, the average prediction time for a package is similar for BERT, RoBERTa, or T5 trained on PyPI, NPM and mixed packages, which is respectively 11.552, 10.030 and 10.527 seconds.

Summary. CEREBRO takes less than two days to train the model; and CEREBRO takes an average of 10.5 seconds to predict whether a package is malicious, which is acceptable.

5.4 Dataset Scale Evaluation (RQ3)

Figure 1 presents the results of our dataset scale evaluation. Overall, CEREBRO *BERT* and CEREBRO *RoBERTa* consistently outperform the state-of-the-art by a significant margin in both PyPI and NPM packages, even with limited dataset scales (i.e., 10%, 20%). In Figure 1(a), CEREBRO *BERT* and CEREBRO *RoBERTa* exhibit similar F1-Scores as the dataset scale changes. CEREBRO *RoBERTa* outperforms *AMALFiDT* by an F1-Score of 0.045 at the 10% scale. As the dataset scale increases, CEREBRO *RoBERTa* further widens the gap, surpassing *AMALFiDT* with an F1-Score of 0.113 at the 100% scale. In Figure 1(b), CEREBRO *BERT*, CEREBRO *RoBERTa*, and CEREBRO *T5* exhibit similar F1-Scores as the dataset scale changes. CEREBRO *RoBERTa* outperforms *AMALFiDT* by an F1-Score of 0.09 at the 10% scale. As the dataset scale increases, CEREBRO *RoBERTa* further keeps the gap, achieving an average F1-Score difference of 0.072 across remaining dataset scales. We also observe that as the training dataset increases, the F1-score improvement for NPM packages is relatively smaller compared to PyPI packages. This is primarily due to the differing levels of package diversity between the two

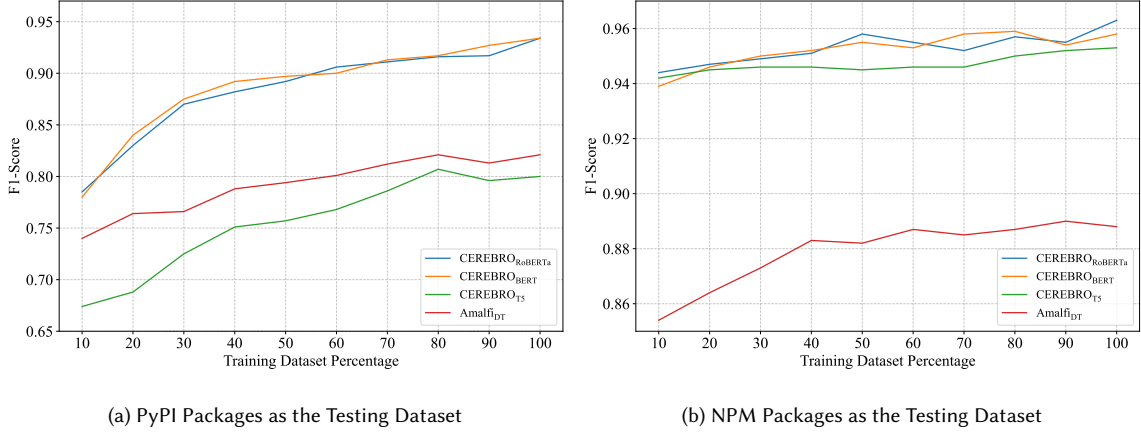


Fig. 1. Effectiveness of CEREBRO $BERT$, CEREBRO $RoBERTa$, CEREBRO T_5 and AMALFI DT in Different Scales of Dataset

Table 7. Evaluation Result on Ablated Versions of CEREBRO (The bold numbers indicate the decreases or increases compared to the non-ablated versions)

Train	Test	CEREBRO w/o Seq						CEREBRO w/o Text						CEREBRO w/ DT	
		BERT		RoBERTa		T5		BERT		RoBERTa		T5		Pre.	Rec.
		Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.	Pre.	Rec.		
PyPI	PyPI	91.3%	87.1%	92.1%	86.2%	92.1%	76.1%	92.6%	85.0%	90.6%	89.6%	94.1%	76.8%	91.4%	89.4%
	PyPI	↓ 4.5%	↓ 2.3%	↓ 3.9%	↓ 5.5%	↓ 1.3%	↑ 5.2%	↓ 3.2%	↓ 4.4%	↓ 5.4%	↓ 2.1%	↑ 0.7%	↑ 5.9%	↓ 4.4%	↓ 0.0%
NPM	NPM	98.0%	92.2%	98.9%	92.1%	98.8%	90.4%	98.1%	90.4%	97.7%	90.4%	99.1%	88.3%	96.6%	92.2%
	NPM	↓ 0.2%	↑ 0.4%	↑ 0.4%	↓ 0.8%	↓ 0.1%	↓ 0.6%	↓ 0.1%	↓ 1.4%	↓ 0.8%	↓ 2.5%	↑ 0.2%	↓ 2.7%	↓ 1.6%	↑ 0.4%
NPM	PyPI	77.3%	36.0%	76.7%	42.1%	64.7%	67.6%	93.7%	18.5%	64.2%	2.0%	88.0%	8.4%	49.5%	11.0%
	PyPI	↑ 5.3%	↓ 13.1%	↑ 1.6%	↓ 2.0%	↓ 0.5%	↑ 4.2%	↑ 21.7%	↓ 30.6%	↓ 10.9%	↓ 42.1%	↑ 22.8%	↓ 55.0%	↓ 22.5%	↓ 38.1%
PyPI	NPM	49.2%	92.2%	38.3%	87.2%	41.3%	86.0%	41.8%	94.0%	28.0%	98.9%	10.6%	13.6%	10.2%	8.4%
	NPM	↑ 7.3%	↓ 0.8%	↓ 8.2%	↓ 5.5%	↑ 3.8%	↓ 4.9%	↓ 0.1%	↑ 1.0%	↑ 18.5%	↑ 6.2%	↑ 26.9%	↓ 77.3%	↓ 31.7%	↓ 84.6%
Mixed	PyPI	92.3%	86.2%	93.4%	88.3%	92.6%	71.3%	95.5%	86.0%	94.7%	87.4%	92.9%	82.0%	90.5%	86.6%
	PyPI	↓ 2.7%	↓ 5.8%	↓ 2.7%	↓ 2.6%	↓ 2.0%	↑ 1.7%	↑ 0.5%	↓ 6.0%	↓ 1.4%	↓ 3.5%	↓ 1.7%	↑ 12.4%	↓ 4.5%	↓ 5.4%
Mixed	NPM	98.2%	91.8%	98.6%	91.7%	98.9%	90.9%	98.7%	91.1%	98.3%	91.1%	98.8%	89.5%	93.9%	92.0%
	NPM	↓ 0.5%	↓ 1.2%	↓ 0.3%	↓ 2.2%	↓ 0.0%	↓ 1.1%	↓ 0.0%	↓ 1.9%	↓ 0.6%	↓ 2.8%	↓ 0.1%	↓ 2.5%	↓ 4.8%	↓ 1.0%

ecosystems. As shown in Fig. 2b, a larger proportion of NPM packages focus on information stealing. Consequently, models trained on NPM packages quickly learn and generalize malicious patterns from smaller datasets than those for PyPI, resulting in better performance at lower data percentages and smaller incremental gains as the dataset scale increases.

Summary. The effectiveness of our approach maintains an advantage over state-of-the-art across different dataset scales. CEREBRO $RoBERTa$ achieves an F1-Score of 0.785 in PyPI and 0.944 in NPM even when the dataset scale is reduced by 90%, indicating the robustness of CEREBRO in situations with scarce data availability.

5.5 Ablation Study (RQ4)

We train the three ablated versions of CEREBRO in the bi-lingual scenario. As shown in Table 7, the ablated version is less effective than the original version. Specifically, the average precision and recall for CEREBRO $BERT$, CEREBRO $RoBERTa$ and CEREBRO T_5 decrease 1.6% and 0.6%, increase 1.6% and decrease 3.7%, decrease 1.4% and 1.9% for CEREBRO w/o Seq in mono-lingual, cross-lingual and bi-lingual scenarios, respectively. Similarly, the average precision and recall decrease 1.4% and 1.2%, decrease 2.0% and 33.0%, decrease 0.5% and 0.7% for CEREBRO w/o Text in mono-lingual, cross-lingual and bi-lingual scenarios, respectively. Further, the average precision and recall decrease 3.0% and 0.2%, 27.1% and 61.4%, 4.7% and 3.2% for CEREBRO w/ DT in mono-lingual, cross-lingual and bi-lingual scenarios, respectively.

Table 8. Result of Real-World Malicious Package Detection

Package Registry	Time Period	Newly Published Package Versions	Flagged by CEREBRO	False Positives	True Positives	Thank Letters	Removed Before Report
PyPI	March	104,512	750	521	229	173	56
	April	101,415	661	560	101	0	101
	May	78,528	422	324	98	30	68
	June	102,088	768	595	173	0	173
	July	101,600	720	670	50	10	40
	August	56,561	298	283	15	14	1
	September	25,193	101	88	13	4	9
	October	29,596	16	12	4	1	3
	Total	599,493	3,746	3,053	683	232	451
NPM	April	130,697	325	178	147	147	0
	May	39,094	145	88	57	35	22
	June	28,764	256	145	111	0	111
	July	34,626	244	155	89	44	45
	August	33,013	514	314	200	131	69
	September	27,350	374	254	120	91	29
	October	30,601	372	297	75	27	48
	Total	324,145	2,230	1431	799	475	324

Summary. Generating behavior sequence, transforming behavior sequence into textual description, and feeding textual description into a classifier fine-tuned from a pre-trained language model are all effective in detecting malicious packages. Averagely, the ablated versions of CEREBRO decrease 4.5% in precision and 11.8% in recall.

5.6 Real-World Usefulness Evaluation (RQ5)

To evaluate the practical usefulness of CEREBRO in analyzing newly published package versions in real-world registries, we design a monitoring system that first monitors and crawls newly published package versions from PyPI and NPM and then runs CEREBRO against the crawled package versions. Our monitoring system has lasted over 8 months for PyPI and 7 months for NPM.

Overall Usefulness Result. In total, CEREBRO scans 923,638 newly published package versions, and flags 3,746 PyPI package versions and 2,230 NPM package versions as potentially malicious, as reported in Table 8. We first manually validate each flagged package version, and confirm 683 PyPI package versions and 799 NPM package versions as true positives. Then, following responsible disclosure process, we promptly report these package versions to administrators of the official PyPI and NPM teams. At the time of reporting, 451 PyPI package versions and 324 NPM package versions have already been removed. The PyPI and NPM teams confirm the remaining 232 PyPI package versions and 475 NPM package versions as malicious, and remove all of them. Thus, we receive 232 and 457 thank letters from PyPI and NPM.

False Positive Analysis. As shown in Table 8, the false positive rate is 81.5% for PyPI and 64.2% for NPM, which is actually high. It is worth mentioning that it takes one of our authors around one hour to manually analyze and confirm the flagged package versions every day, which is still acceptable. After analyzing all the false positives, we identify four major types of benign packages that are flagged as malicious, as their behavior sequences are similar to malicious ones.

First, some benign packages encapsulate RESTful APIs, which enable data transmission to remote servers. Their behavior sequences consist of features about *Information Reading* and *Data Transmission* (e.g., R1, D1, D2 and D3). Second, some benign packages encapsulate CLI commands into Python/JavaScript methods where their behavior sequences resemble those malicious packages in using features about *Payload Execution*. Third, some benign packages behave similarly to Trojans, but their payloads serve benign purposes. Normally, their behavior sequences resemble Trojans by starting with D1 and D3, then undertaking intermediate actions such as R1 and R4, and ending with P2 and P3. Last, some benign packages consist of a single meta utility module that incorporates features ranging from *Information*

Table 9. Result of Incremental Learning

Added Train (#)	Time Period	New Test (#)	Pre.
None	-		7.2%
PyPI (750)	March	PyPI (1,135)	26.5%
Mixed (1,736)	March-April		36.5%
Mixed (2,303)	March-May		39.0%
Mixed (3,327)	March-June		58.2%
None	-		32.2%
PyPI (750)	March	NPM (1,504)	76.3%
Mixed (1,736)	March-April		80.3%
Mixed (2,303)	March-May		80.2%
Mixed (3,327)	March-June		80.7%

Reading, Data Transmission, Encoding to Payload Execution. It causes CEREBO to recognize parts of behavior sequences as similar to those of malicious packages.

False Negative Analysis. To measure false negatives of CEREBO, we reached out to package registries. However, PyPI replied that they did not maintain a malicious package list and NPM refused to share the list. Therefore, we try to use state-of-the-art detectors to identify true positives. Specifically, we employ AMALFI [42] to identify potentially malicious packages during the same monitoring time period. It flagged 41,434 PyPI package versions and 1,930 NPM package versions as potentially malicious. Then, we undertake a manual inspection process with a sampling rate of 95% confidence level and 5% error margin, which results in 381 PyPI packages and 164 NPM packages. After manual validation, we identify 9 malicious PyPI packages and 45 malicious NPM packages. Remarkably, only 2 PyPI packages and 5 NPM packages went undetected by CEREBO. It indicates that the false negative rate of CEREBO is relatively low in real-world detection.

Incremental Learning Result. As we are continuously confirming and accumulating true positives and false positives of malicious packages based on our monitoring system, one potential way to reduce false positives is to adopt incremental learning on such true positives and false positives. To this end, we add the true positives and false positives confirmed during our monitoring from March to June as two incremental datasets for retraining. In particular, we add 750 packages from March, 1,736 packages from March to April, 2,303 packages from March to May and 3,327 packages from March to June, as reported in the third to sixth row, and eight to eleven row under the column of *Added Train* in Table 9. Note that the added training dataset for March contains only PyPI packages because monitoring NPM packages starts from April. Then, we retrain four models of CEREBO by augmenting the original mixed training dataset with the new mixed PyPI and NPM packages. Finally, we select true positives and false positives confirmed during our monitoring from July to October as the new testing dataset. In particular, there are 1,135 PyPI packages and 1,504 NPM packages as reported in the third column in Table 9.

Overall, by incorporating new malicious and benign packages into the training dataset, the precision of CEREBO improves significantly. Specifically, when CEREBO is retrained with new mixed packages, the precision of detecting malicious PyPI packages reaches 58.2%, while the precision of detecting malicious NPM packages reaches 80.7%, achieving a precision increase of 51.0% for PyPI packages and 48.5% for NPM packages in total.

Malicious Package Characteristics. For each of the 683 PyPI and 799 NPM malicious package versions which are previously unknown, we manually look into their malicious behavior to understand their characteristics. On the one hand, we determine their triggering scenarios, and report the result in Figure 2a. We can observe that 477 (69.8%) of the malicious PyPI package versions and 758 (94.9%) of the malicious NPM package versions trigger their malicious behavior at install-time. This is reasonable because install-time execution can achieve the highest probability of successfully carrying out attacks. Moreover, a small number of malicious PyPI package versions opt to trigger malicious behavior other than install-time, accounting for 161 (23.6%) at import-time and 45 (6.6%) at run-time. Noticeably, there are only 17

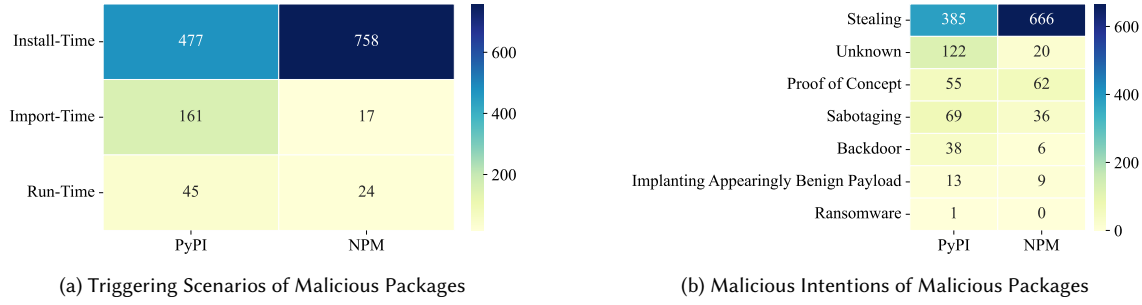


Fig. 2. Triggering Scenarios and Malicious Intentions of New Malicious Packages

(2.1%) of the malicious NPM package versions that adopt import-time triggering, and 24 (3.0%) malicious NPM package versions adopt run-time triggering.

On the other hand, we explore the intentions of malicious package versions by executing them in a sandbox, and present the result in Figure 2b. Generally, the malicious intentions include stealing (e.g., personal data or credentials), sabotaging (e.g., shutting down firewalls), proof of concept (which is harmless but to demonstrate something malicious can be done), backdoor (i.e., enabling remote-controlled operations for an attack to exploit an infected machine), implanting appearingly benign payload (e.g., the benign payload might change to be malicious via morphing Trojan), and ransomware. Besides, we fail to execute the payload for 142 of the malicious package versions due to non-existent URLs or run-time exceptions, and thus we label their malicious intentions as unknown.

Summary. CEREBRO detects 683 and 799 previously unknown malicious PyPI and NPM package versions. All of them have been removed by the official PyPI and NPM teams, and we also receive 707 thank letters from them. Further, by incorporating new malicious and benign packages, CEREBRO learns new knowledge to improve its precision. Besides, most (83.3%) of these new malicious package versions execute the malicious behavior at install-time; and most (70.9%) of them attempt to steal sensitive information.

5.7 Threats and Limitations

Threats. First, as the campaign against malware continues, new strains of malicious packages emerge daily. This implies that the original dataset of malicious packages used in our evaluation may not encompass the full spectrum of new malicious packages, hindering CEREBRO’s effectiveness. However, we have demonstrated the effectiveness of CEREBRO in learning from newly published packages and adapting to detect emerging threats. Second, while we thoroughly examine the potentially malicious package versions identified by CEREBRO, we acknowledge that the vast workload makes it impractical to validate every potentially benign package version. Therefore, we only systematically report the result of precision but not the result of recall. Third, CEREBRO can generate false positives. To address this, we conduct manual validation on identified package versions before reporting them to PyPI and NPM teams. While this process can also introduce false positives, the fact that the reported packages are subsequently removed indicates the validity of our validation. Fourth, as pre-trained language models continue to advance rapidly, we expect to see new and more powerful models. Nevertheless, the choice of language models is orthogonal to our approach. We plan to test the effectiveness of additional language models on this task. Fifth, the varying behavior sequences between malicious and benign packages may reduce the accuracy of CEREBRO in cross-lingual scenarios. Nevertheless, the cross-lingual setting represents an extreme case in real-world where there are no malicious samples within the same ecosystem.

Limitations. First, our method is designed for PyPI and NPM packages. However, our method can be easily generalized into other ecosystems by implementing the feature extractor and behavior sequence generator for a new ecosystem. The feature set is language-agnostic. By leveraging the existing AST parser and call graph generator of the new ecosystem, we believe that the integration can be straightforward. Second, large packages may have a large size of the behavior sequence which can exceed the 512 token limit of BERT. However, based on our analysis of packages in Table 8, we identify a relatively modest proportion (13.6%) of packages that exceed the limit. We plan to split the behavior sequence into multiple segments to circumvent the limit while maintaining their sequential integrity. Third, the proposed approach encounters higher false positives in real-world setting. We foresee several challenges and opportunities that could help reduce false positive rates. On the one hand, we can leverage ensemble methods. Utilizing a combination of different detection tools could be advantageous. An ensemble approach, where decisions are based on the consensus among multiple tools, might improve accuracy and reduce false positive rates. On the other hand, we can utilize dynamic analysis during our detection. Incorporating dynamic analysis into the detection process could provide deeper insights into the behavior of packages by capturing detailed runtime behaviors, potentially reducing false positives by confirming suspicious activities through actual execution rather than static analysis. Nonetheless, with a large number of new packages being released daily, it remains an open question how to leverage the benefits of dynamic analysis while mitigating its computational and time costs.

6 RELATED WORK

Malicious Package Detection. Recently, there has been an increasing number of OSS supply chain attacks that inject malicious code into packages, with package registries such as NPM and PyPI being the largest targets [27, 34]. Ohm et al. [37] were the first to systematically investigate malicious packages. They collected 174 real-world malicious packages from NPM, PyPI and RubyGems. Using this dataset, they manually analyzed how malicious behavior was injected (e.g., via typosquatting) and triggered (e.g., on installation), what the objective (e.g., data exfiltration) of malicious behavior was, and whether obfuscation was employed. Similarly, Guo et al. [54] conducted an empirical study to understand the characteristics of the malicious code lifecycle in the PyPI ecosystem. Various approaches have also been proposed to detect malicious packages, mostly in NPM and PyPI.

In the NPM ecosystem, Zahan et al. [59] identified six signals of security weakness in NPM supply chain, and proposed a rule-based malicious package detector that searches for keywords in install scripts. Garrett et al. [11] selected features about whether libraries that access the network, file system and operating system processes are used, whether code is evaluated at runtime, and whether new files, new dependencies and new hookup script entries are present, and adopted clustering to build a benign behavior model which is used to detect malicious NPM packages. Differently, Ohm et al. [36] adopted AST (abstract syntax tree)-level clustering to establish a malicious behavior model. Instead of using unsupervised methods, Fass et al. [8, 9] built a random forest classifier based on the frequency of specific patterns extracted from AST, CFG (control flow graph) and PDG (program dependency graph) of benign and malicious JavaScript samples. This approach is specifically designed for obfuscated JavaScript programs, and thus could be ineffective for non-obfuscated ones. Ohm et al. [35] extended the feature set of Garrett et al.’s work [11] by including features about package metadata and obfuscation. Similarly, Sejfia et al. [42] proposed AMALFI, which uses an extended feature set from Garrett et al.’s work [11] to train classifiers. These approaches capture malicious behavior as discrete features, hindering their accuracy in detecting malicious packages. Differently, we model malicious behavior in a sequential manner, which can model the maliciousness more precisely. Notice that Ferreira et al. [10] proposed a

lightweight permission system to protect applications from malicious package updates at runtime. Recently, Huang et al. [19] proposes SpiderScan to identify malicious NPM packages based on graph-based behavior modeling and matching.

In the PyPI ecosystem, Vu et al. [52] proposed a rule-based approach to detect malicious PyPI packages that are spread by typosquatting and combosquatting attacks. The rule checks whether the package has the name similar to a Python standard library or a package with known source code repository. Besides, there are several other rule-based detectors, e.g., Bandit4Mal [48] and Malware Checks [53]. Vu et al. [50] conducted a comprehensive evaluation of these detectors. Instead of relying on rules, Vu et al. [49, 51] identified malicious packages based on discrepancies between the source code repository and distributed artifact of a Python package. Liang et al. [29] used both package metadata features and code features to learn an isolated forest model for detecting malicious packages. Recently, Liang et al. [55] utilized clustering techniques to group the PyPI installation scripts so that outliers can be identified. Sun et al. [43] proposes to integrate deep code behavior features with metadata features to detect malicious PyPI packages. These approaches share the same limitation with those detectors in NPM because they also model malicious behavior as discrete features.

Moreover, some approaches support different ecosystems. OSS Detect Backdoor [33] is a rule-based detector that support 15 ecosystems. Taylor et al. [46] leveraged package name similarity and package popularity to detect malicious packages caused by typosquatting in NPM, PyPI and RubyGems. Ladisa et al. [26] utilized language-independent feature (e.g. number of URLs, entropy of strings in code files, etc.) to detect malicious packages in NPM and PyPI. However, these approaches are not based on high-level semantics of source code, which can produce high false positives. Different from the simple rule in Taylor et al. and Ladisa et al., Duan et al. [6] derived five metadata analysis rules, four static analysis rules and four dynamic analysis rules to detect malicious packages in NPM, PyPI and RubyGems. However, it is heavyweight as it relies on program analysis.

Squatting attacks are also common in other domains, e.g., domain name system [1, 22, 23, 44, 45], container registry [31] and mobile app [17]. Malware detection has been widely studied [54, 58]. OSS opens up new opportunities for malware detection due to the availability of source code.

Security Threats in OSS Supply Chain. Apart from malicious packages, there exist various security threats in OSS supply chain. Several studies [3, 5, 16, 20, 59] have been conducted to investigate security threats in package registries such as NPM and PyPI. Ladisa et al. [25] reported a taxonomy of attacks on all OSS supply chain stages from code contributions to package distribution. They also assessed the safeguards against OSS supply chain attacks. Koishybayev et al. [24] and Gu et al. [15] characterized security threats in continuous integration workflows that produce OSS.

Enck and Williams [7] summarized five challenges in OSS supply chain security, with the top one being updating vulnerable dependencies. Specifically, the vulnerability impact analysis along the supply chain has been widely explored in various ecosystems, e.g., NPM [30, 60, 61], Maven [12, 18, 39], PyPI [2, 39, 41] and RubyGems [39, 60]. To cope with vulnerable dependencies, various software composition analysis tools have been proposed [18, 30, 38, 57]. Besides, several advances have been made on detect maliciousness in different stages of software development. Goyal et al. [14] and Gonzalez et al. [13] proposed to detect malicious commits on GitHub. Cao and Dolan-Gavitt [4] tried to identify malware in GitHub forks. Lamb and Zacchiroli [28] and Wheeler [56] attempted to avoid the injection of maliciousness in compilation by ensuring reproducible builds.

7 CONCLUSION AND FUTURE WORK

XXXX

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (Grant No. 62332005, 62372114 and 62402342).

REFERENCES

- [1] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months' worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium*.
- [2] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. 2023. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering* 28 (2023).
- [3] Aadesh Bagmar, Josiah Wedgwood, Dave Levin, and Jim Purtilo. 2021. I know what you imported last summer: A study of security threats in the python ecosystem. *arXiv preprint arXiv:2102.06301* (2021).
- [4] Alan Cao and Brendan Dolan-Gavitt. 2022. What the Fork? Finding and Analyzing Malware in GitHub Forks. In *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web*.
- [5] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security*. 565–574.
- [6] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. In *Proceedings of 27th Annual Network and Distributed System Security Symposium*.
- [7] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.
- [8] Aurore Fass, Michael Backes, and Ben Stock. 2019. Jstap: A static pre-filter for malicious javascript detection. In *Proceedings of the 35th Annual Computer Security Applications Conference*. 257–269.
- [9] Aurore Fass, Robert P Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 303–325.
- [10] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in npm with a lightweight permission system. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*. 1334–1346.
- [11] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*. 13–16.
- [12] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2021. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. *Journal of Systems and Software* 172 (2021).
- [13] Danielle Gonzalez, Thomas Zimmermann, Patrice Godefroid, and Max Schäfer. 2021. Anomalous: Automated detection of anomalous and potentially malicious commits on github. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice*. 258–267.
- [14] Raman Goyal, Gabriel Ferreira, Christian Kästner, and James Herbsleb. 2018. Identifying unusual commits on GitHub. *Journal of Software: Evolution and Process* 30, 1 (2018).
- [15] Yacong Gu, Lingyun Ying, Huajun Chai, Chu Qiao, Haixin Duan, and Xing Gao. 2023. Continuous Intrusion: Characterizing the Security of Continuous Integration Services. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1561–1577.
- [16] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2022. Investigating Package Related Security Threats in Software Registries. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1151–1168.
- [17] Yangyu Hu, Haoyu Wang, Ren He, Li Li, Gareth Tyson, Ignacio Castro, Yao Guo, Lei Wu, and Guoai Xu. 2020. Mobile app squatting. In *Proceedings of The Web Conference 2020*. 1727–1738.
- [18] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.
- [19] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [20] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and pypi registries. In *Proceedings of the Second International Workshop on Deployable Machine Learning for Security Defense*. 132–156.
- [21] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.
- [22] Mohammad Taha Khan, Xiang Huo, Zhou Li, and Chris Kanich. 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. 135–150.
- [23] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on*

- Computer and Communications Security*. 569–586.
- [24] Igbek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of Github (CI) Workflows. In *Proceedings of the 31st USENIX Security Symposium*. 2747–2763.
 - [25] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1509–1526.
 - [26] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 71–82.
 - [27] Ravie Lakshmanan. 2022. Malware Strains Targeting Python and JavaScript Developers Through Official Repositories. Retrieved May 30, 2023 from <https://thehackernews.com/2022/12/malware-strains-targeting-python-and.html>
 - [28] Chris Lamb and Stefano Zacchiroli. 2021. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software* 39, 2 (2021), 62–70.
 - [29] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*. 606–613.
 - [30] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684.
 - [31] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022. Exploring the Uncharted Space of Container Registry Typosquatting. In *Proceedings of the 31st USENIX Security Symposium*. 35–51.
 - [32] Yinhao Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
 - [33] Microsoft. 2020. OSS Detect Backdoor. Retrieved May 30, 2023 from <https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor>
 - [34] Phil Muncaster. 2023. Hundreds Malicious Packages NPM. Retrieved May 30, 2023 from <https://www.infosecurity-magazine.com/news/hundreds-malicious-packages-npm/>
 - [35] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–10.
 - [36] Marc Ohm, Lukas Kempf, Felix Boes, and Michael Meier. 2020. Supporting the detection of software supply chain attacks through unsupervised signature generation. *arXiv preprint arXiv:2011.02235* (2020).
 - [37] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 23–43.
 - [38] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*. 449–460.
 - [39] Gede Artha Azriadi Prana, Abhishek Sharma, Lwin Khin Shar, Darius Foo, Andrew E Santosa, Asankhaya Sharma, and David Lo. 2021. Out of sight, out of mind? How vulnerable dependencies affect open-source projects. *Empirical Software Engineering* 26 (2021).
 - [40] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
 - [41] Jukka Ruohonen, Kalle Hjerpe, and Kalle Rindell. 2021. A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI. In *Proceedings of the 18th International Conference on Privacy, Security and Trust*. 1–10.
 - [42] Adriana Seifia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*. 1681–1692.
 - [43] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
 - [44] Janos Szurdi, Balazs Kocso, Gabor Cseh, Jonathan Spring, Mark Felegyhazi, and Chris Kanich. 2014. The long “tail” of typosquatting domain names. In *Proceedings of the 23rd USENIX Security Symposium*. 191–206.
 - [45] Rashid Tahir, Ali Raza, Faizan Ahmad, Jehangir Kazi, Fareed Zaffar, Chris Kanich, and Matthew Caesar. 2018. It’s all in the name: Why some URLs are more vulnerable to typosquatting. In *Proceedings of the 2018 IEEE Conference on Computer Communications*. 2618–2626.
 - [46] Matthew Taylor, Rituraj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*. 112–131.
 - [47] virustotal. 2023. Virustotal. Retrieved May 1, 2023 from <https://www.virustotal.com/gui/home/upload>
 - [48] Duc-Ly Vu. 2020. Bandit4Mal. Retrieved May 30, 2023 from <https://github.com/lyvd/bandit4mal>
 - [49] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. Lastpymile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 780–792.
 - [50] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *Proceedings of the 45th International Conference on Software Engineering*.
 - [51] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Towards using source code repositories to identify software supply chain attacks. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 2093–2095.

- [52] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020. Typosquatting and combosquatting attacks on the python ecosystem. In *Proceedings of the 2020 IEEE European Symposium on Security and Privacy Workshops*. 509–514.
- [53] Warehouse. 2020. Malware Checks. Retrieved May 1, 2023 from <https://warehouse.pypa.io/development/malware-checks.html>
- [54] Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
- [55] Liang Wentao, Ling Xiang, Wu Jingzheng, Luo Tianyue, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
- [56] David A Wheeler. 2005. Countering trusting trust through diverse double-compiling. In *Proceedings of the 21st Annual Computer Security Applications Conference*. 33–48.
- [57] Meiqiu Xu, Ying Wang, Shing-Chi Cheung, Hai Yu, and Zhiliang Zhu. 2022. Insight: Exploring Cross-Ecosystem Vulnerability Impacts. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- [58] Yanfang Ye, Tao Li, Donald Adjeroh, and S Sitharama Iyengar. 2017. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)* 50, 3 (2017), 1–40.
- [59] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.
- [60] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering* 27 (2022).
- [61] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*. 995–1010.